

Redesign Deck Picker
AnkiDroid Proposal
2026

Shaan Narendran
Github: ShaanNarendran
Discord: shaan_naren

Table of Contents

Table of Contents	2
Requirements	3
My contributions to AnkiDroid	3
My project	4
Redesign Deck Picker (175 hours)	4
Why I want to work on it	4
1. Bottom Navigation Menu XML Layouts	4
2. Bottom Navigation Bar Base Class	7
3. Add Actions	9
4. More button	12
5. Fragment Migration	13
6. Decoupling the Navigation Drawer	16
7. Reviewer Back Button	18
8. Card Browser & Statistics Design	19
9. Keyboard Shortcuts	21
10. Tablet UI	22
11. Deck Picker Redesign	25
Success Criteria	31
Timeline	31
Myself:	33
My personal experience with Anki/AnkiDroid	33
What I expect to get from participating in GSoC	33
Programming languages and tools I know	33
Example of skills you taught yourself	33
Anything else	33

Requirements

By submitting this GSoC proposal, I confirm that I:

- [✓] Meet [Google's Eligibility Requirements](#)
- [✓] Have installed and used AnkiDroid
- [✓] Have a basic understanding of Anki's concepts such as decks, cards, notes, note types, new cards, review cards and cards in learning
- [✓] Have access to a physical Android device
- [✓] Have access to an internet connection throughout GSoC. At least enough to communicate via text, update Android Studio when required, and interact with GitHub
- [✓] Have at least 175 hours to dedicate to the GSoC, and have time for at least one one-to-one meeting with a mentor each working week, to discuss progress and future tasks
- [✓] Have been able to compile AnkiDroid, install it on a device, run automated tests and [enable WebView Debugging](#)
- [✓] Have sufficient English reading & writing abilities to participate in technical discussions
- [✓] Have some experience with Kotlin programming, testing, debugging, and git

My contributions to AnkiDroid:

- Link to a merged AnkiDroid Pull Request (PR) I created that contains at least one unit test¹ :
 - <https://github.com/ankidroid/Anki-Android/pull/20176>
- Link to a merged AnkiDroid PR I created that contains at least one code change that is not a test:
 - <https://github.com/ankidroid/Anki-Android/pull/19730>
 - <https://github.com/ankidroid/Anki-Android/pull/19980>
 - <https://github.com/ankidroid/Anki-Android/pull/20131>
 - <https://github.com/ankidroid/Anki-Android/pull/20176>
- Link to an AnkiDroid PR I constructively² commented on that I did not create:
 - <https://github.com/ankidroid/Anki-Android/pull/20308>
 - <https://github.com/ankidroid/Anki-Android/pull/20258>

¹ Look at <https://github.com/ankidroid/Anki-Android/wiki/Testing> if you are new to unit tests.

² A comment who helped improve the PR. E.g. stating that something is not clear and requires comments/javadoc/detailed commit message, suggesting an improvement to the change,

My project:

Redesign Deck Picker (175 hours)

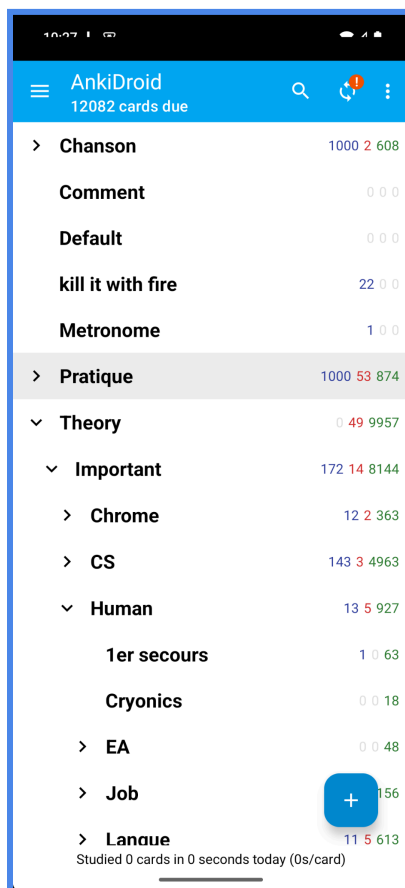
AnkiDroid has had the same base UI for a long, long time. As other apps moved on to newer themes and layouts, AnkiDroid has been stuck in time and this project aims to change things by redesigning the UI by adding a bottom navigation bar along with some more additions to be compliant to the latest android material guidelines and give the app a modern feel.

I want to work on it because:

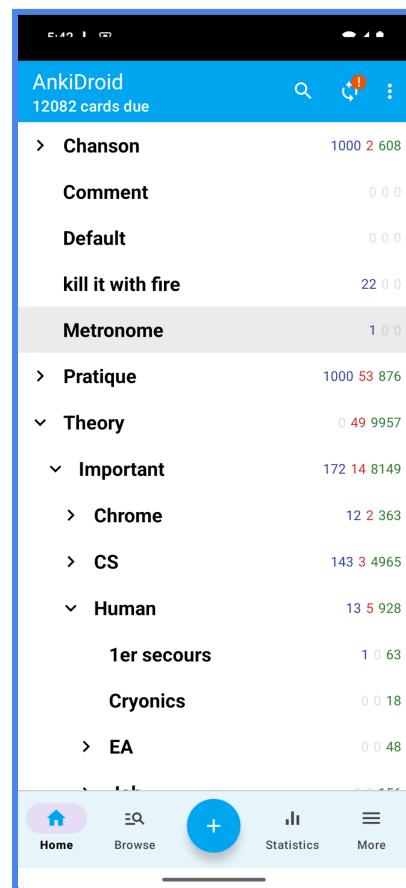
I believe that this is a super impactful project, obviously the first thing the user sees is the deck picker screen and being able to redesign such an important part of the app, while it is a lot of responsibility, sounds like an amazing challenge to me. I've grown up watching study videos that all mention Anki so to now be able to make such a huge change to an app used by millions is a dream come true.

How I will solve it/make progress:

1. Bottom Navigation Menu XML Layouts



Before



After

As we can see above, the current deck picker screen has none of the most important destinations visible to the user at one glance, since destinations like the card browser and statistics are hidden by the navigation drawer, the user has to use 2 hands to access these tabs. In most modern apps, bottom navigation bars have become the standard because they allow for users to easily access important tabs/destinations using a single hand.

(i) The first proposed change is to instantiate the XML layouts for the bottom navigation bar and the new centered add button³, these are the most important layouts for the redesign and will be the starting point for the project.

Bottom_nav_menu.xml

This xml defines the 5 buttons for the navigation bar - Home, Browse, Add (space given here since it's to be moved to this space), Statistics, More

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item
        android:id="@+id/action_home"
        android:icon="@drawable/home_icon"
        android:title="Home" />

    <item
        android:id="@+id/action_card_browser"
        android:icon="@drawable/ic_card_browser"
        android:title="Browse" />

    <item
        android:id="@+id/action_add"
        android:enabled="false"
        android:title="" />

    <item
        android:id="@+id/action_stats"
        android:icon="@drawable/ic_bar_chart_black"
        android:title="@string/statistics" />

    <item
        android:id="@+id/action_more"
        android:icon="@drawable/ic_menu_24"
        android:title="More" />

</menu>
```

³ Referred to the youtube homepage on android, they use a flat centered add button

activity_bottom_nav.xml

We can create a wrapper xml that will wrap all our xml destinations with the bottom nav bar to completely avoid duplicated code, how this is implemented is detailed in the next section.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/bottom_nav_root"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Container for subclass activity content -->
    <FrameLayout
        android:id="@+id/bottom_nav_content_container"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginBottom="80dp" />

    <!-- Thin divider above bottom nav -->
    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:layout_gravity="bottom"
        android:layout_marginBottom="80dp"
        android:background="?attr/colorOutlineVariant" />

    <!-- Bottom Navigation Bar -->
    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        android:layout_width="match_parent"
        android:layout_height="80dp"
        android:layout_gravity="bottom"
        android:elevation="0dp"
        android:background="?attr/colorSurface"
        app:menu="@menu/bottom_nav_menu"
        app:labelVisibilityMode="labeled"
        app:itemIconTint="@color/bottom_nav_icon_color" />

    <!-- Centered Add FAB -->
    <include
        android:id="@+id/floating_action_button"
        layout="@layout/floating_add_button"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

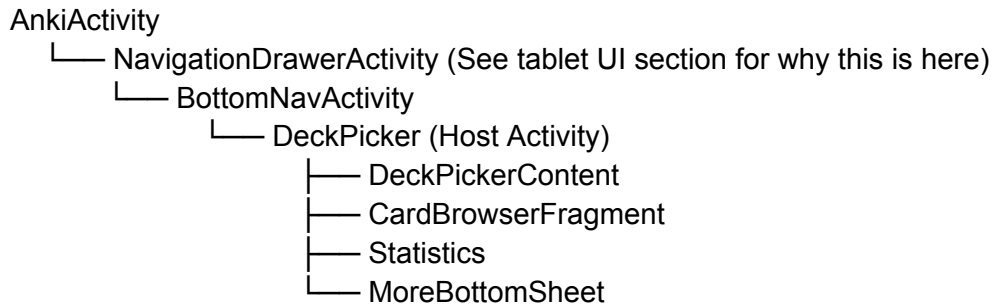
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

For measurement reference: <https://m3.material.io/components/navigation-bar/specs>

2. Bottom Navigation Bar Base Class

The next step would be to create a bottom nav bar base class (BottomNavActivity) which will store the logic for all the routing and inflation of the bottom navigation bar. Here, DeckPicker will be our main host activity and it swaps out the fragments using show() and hide().

The structure below is what the new inheritance would look like:



Implementation:

The base class is meant to contain all the logic required for the bottom navigation bar. These are:

→ View inflation: We don't want to duplicate code in all the xml for the destination pages so instead we can use activity_bottom_nav.xml as a coordinator layout that contains the destination xml's along with the nav bar implementation. This intercepts the setContentView() calls.

Kotlin snippet:

```
override fun setContentView(layoutResID: Int) {
    val childView = inflater.inflate(layoutResID, null)
    setContentView(childView)
}

override fun setContentView(view: View?) {
    val root = inflater.inflate(R.layout.activity_bottom_nav,
    null) as androidx.coordinatorlayout.widget.CoordinatorLayout
    val container =
    root.findViewById<android.widget.FrameLayout>(R.id.bottom_nav_content_conta
    iner)
    if (view != null) container.addView(view)
    super.setContentView(root)
}
```

→ Destination Functions: To handle the destination taps, we use the fragment manager to manage our CardBrowser and Statistics Fragments via using the show() and hide() methods to hide the fragment not in use. This allows for smooth switching while also keeping the state and memory preserved as mentioned in M3 guidelines.

Kotlin snippet (in DeckPicker.kt):

```
override fun onBrowserSelected() {
    deckPickerContentView.visibility = View.GONE
    supportActionBar?.hide()
    showFragment(CardBrowserFragment(), "browser")
}

override fun onStatsSelected() {
    deckPickerContentView.visibility = View.GONE
    supportActionBar?.hide()
    showFragment(Statistics(), "stats")
}

override fun onHomeSelected() {
    hideCurrentFragment()
    deckPickerContentView.visibility = View.VISIBLE
    supportActionBar?.show()
}
```

→ Icon helpers: Right now, the destinations contain the navigation drawer icon, this needs to be replaced by the back icon so we can define helper functions for the same:

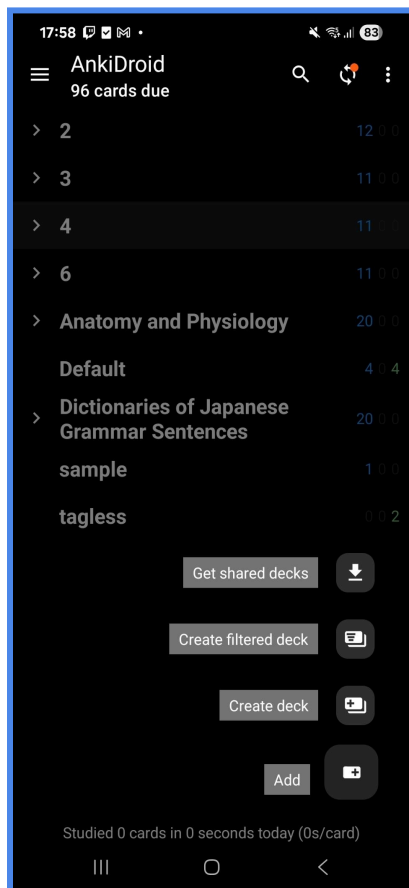
Kotlin Snippet:

```
protected fun showBackIcon() {
    supportActionBar?.setDisplayHomeAsUpEnabled(true)
    supportActionBar?.setHomeButtonEnabled(true)
}
```

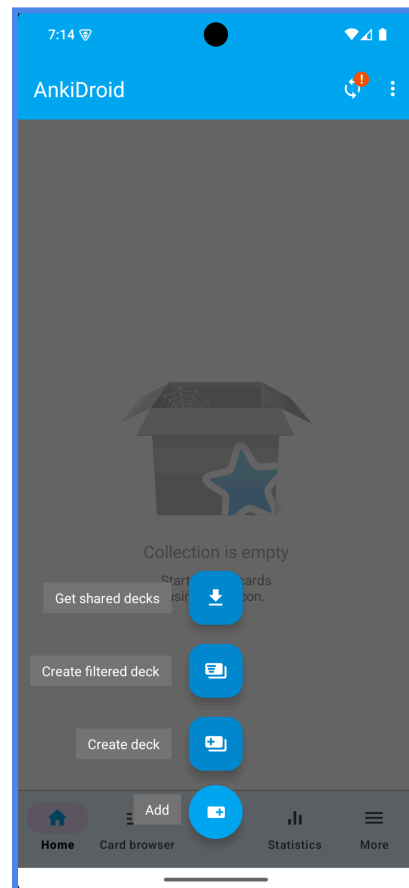
```
protected fun hideNavigationIcon() {
    supportActionBar?.setDisplayHomeAsUpEnabled(false)
    supportActionBar?.setHomeButtonEnabled(false)
}
```

→ Fragment-Owned Toolbars: Because DeckPicker acts as the host window, leaving its Action Bar visible while swapping fragments would cause graphical glitches (as Statistics and CardBrowserFragment draw their own toolbars). By using `supportActionBar?.hide()` in the destination routing logic, we can cleanly delegate all Action Bar management to the individual Fragments, this prevents overlaps.

3. Add Actions



Before



After

The current add fab, located in `DeckPickerFloatingActionMenu.kt`, implements the button in such a way that when pressed it opens to 4 new fab buttons that provide the options to the user. We can use this existing implementation but center it in our navigation bar to allow for a smooth migration. Currently, the layout for the add fab button is present in `floating_add_button.xml`, the issue is that we can't directly set the positioning of the button to the center since the animations present in the `DeckPickerFloatingActionMenu.kt` are done in the assumption that the button is in the bottom left.

To move the add fab to the center, there are changes to make in both the animation algorithms as well as the XML layouts:

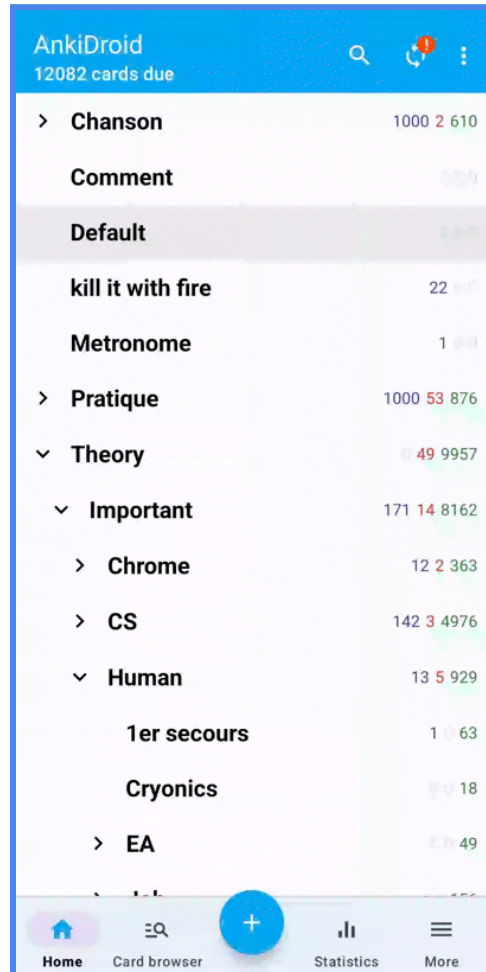
(i) Currently if the add fab button is pressed, the buttons animate up into a column and are present in the right side of the screen. If we try moving this to the center navigation bar by changing the layout to `center_horizontal`, we will see that the buttons are not aligned. This is due to the fact that `LinearLayouts` wraps the contents of each row and since they have different lengths, one row would be centered differently to another row.

To fix this, we can apply a `RelativeLayout` to each row and set the width to `match_parent`. This effectively decouples the width of each row with the positioning of the button so that every button can be centered. For positioning the buttons, we can use `layout_centerHorizontal="true"` so they are centered on the screen and for placing the labels, we can use `layout_toStartOf`.

XML Snippet:

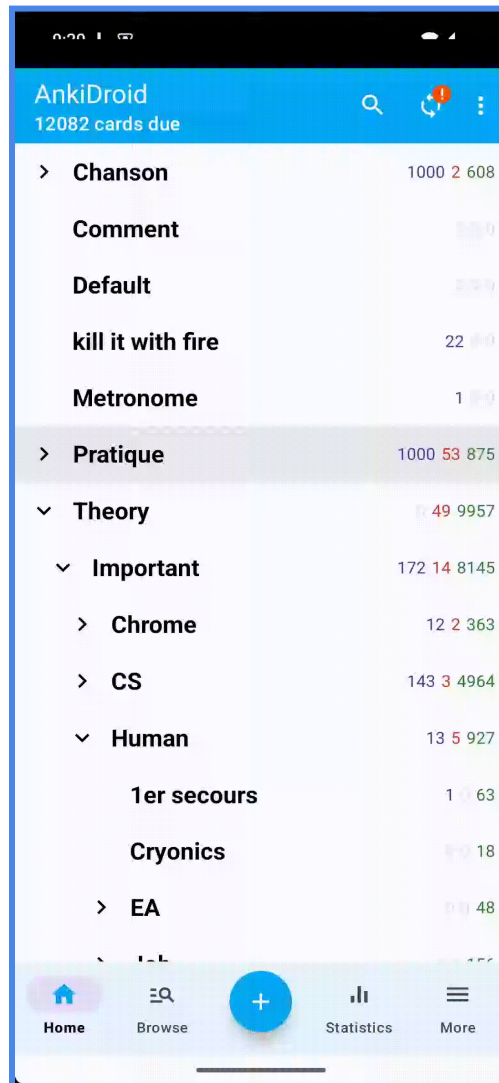
```
<RelativeLayout
    android:id="@+id/add_shared_layout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:clipToPadding="false"
    android:visibility="gone"
    android:alpha="0"
    android:translationY="300px">
    <TextView
        android:id="@+id/add_shared_label"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toStartOf="@id/add_shared_button"
        android:layout_centerVertical="true"
        android:layout_marginEnd="5dp"
        android:paddingTop="5dp"
        android:paddingBottom="5dp"
        android:paddingStart="10dp"
        android:paddingEnd="10dp"
        android:background="@color/material_grey_600"
        android:textColor="@color/white"
        android:text="@string/menu_get_shared_decks"
        android:focusable="true"
        android:clickable="true" />
    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/add_shared_button"
        style="?attr/floatingActionButtonSmallStyle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:layout_margin="10dp"
        app:backgroundTint="?attr/fab_normal"
        android:src="@drawable/ic_file_download_white"
        android:contentDescription="@string/menu_get_shared_decks"
        android:focusable="true"
        android:clickable="true" />
</RelativeLayout>
```

(ii) There are small changes we can make to the animations and padding to get it to fit properly at the center of the screen without having any visual hitches.



4. More button

Since the bottom navigation bar will be replacing the navigation drawer, the destinations that were present there have to be moved to the more tab. Settings, manage note types, help, and the support AnkiDroid pages are not destinations that the user would need to visit frequently and hence they can be present in the new more bottom dialog sheet.



Another advantage to having more tabs in the bottom navigation bar is that now users can access all the destinations using only one hand as opposed to having to use 2 before.

The implementation for the above is very similar to that of the add actions implementation. We use the same BottomSheetBehaviour import which handles all the animations, screen dimming, swiping etc implicitly. We can directly wire the respective activities using their open functions.

5. Fragment Migration

We need to migrate statistics and card browser to fragments to allow for the architecture we decided. The implementation would be as follows:

(i) Statistics is already in a fragment that loads the webview to load content, but it launches via a single fragment activity (found in StatisticsDestination), we need to change this in our onStatsSelected() implementation to show and hide the fragment.

```
override fun onStatsSelected() {  
    deckPickerContentView.visibility = View.GONE  
    showFragment(Statistics(), "statistics")  
}
```

The tests found in StatisticsTest also need to be updated to launch deckpicker and use the statistics fragment.

(ii) The card browser migration has been started in [PR#18463](#) and the menu was further migrated in [PR #20211](#), both of the PRs migrated most of the core functionalities of cardbrowser to the fragment i.e. the list UI and the search bar. There are a few functionalities left to convert which are:

1. Result launchers: These are still present in the main cardbrowser activity but can be moved over to the fragment since they use the registerForActivityResult() API which allows fragments to call on an activity and wait for the result without needing to be done in the parent activity. The launchers we need to move are onEditCardActivityResult, onAddNoteActivityResult, and onPreviewCardsActivityResult

Current code in CardBrowser:

```
private var onEditCardActivityResult =  
    registerForActivityResult(StartActivityForResult()) { result:  
ActivityResult ->  
        Timber.i("onEditCardActivityResult: resultCode=%d",  
result.resultCode)  
        if (result.resultCode == DeckPicker.RESULT_DB_ERROR) {  
            closeCardBrowser(DeckPicker.RESULT_DB_ERROR)  
            return@registerForActivityResult  
        }  
        result.data?.let {  
            if (  
  
it.getBooleanExtra(NoteEditorFragment.RELOAD_REQUIRED_EXTRA_KEY, false) ||  
  
it.getBooleanExtra(NoteEditorFragment.NOTE_CHANGED_EXTRA_KEY, false)  
                ) {
```

```

        forceRefreshSearch()
    }
}

invalidateOptionsMenu()
if (result.resultCode == RESULT_OK) {
    viewModel.onCurrentNoteEdited()
}
}

```

Code after migration:

```

private val onEditCardActivityResult = registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()
) { result ->
    Timber.i("onEditCardActivityResult: resultCode=%d", result.resultCode)

    if (result.resultCode == DeckPicker.RESULT_DB_ERROR) {
        requireActivity().setResult(DeckPicker.RESULT_DB_ERROR)
        requireActivity().finish()
        return@registerForActivityResult
    }

    result.data?.let { intentData ->
        if (

intentData.getBooleanExtra(NoteEditorFragment.RELOAD_REQUIRED_EXTRA_KEY,
false) ||

intentData.getBooleanExtra(NoteEditorFragment.NOTE_CHANGED_EXTRA_KEY,
false)

        ) {
            searchViewModel.executeSearch()
        }
    }

    requireActivity().invalidateOptionsMenu()
    if (result.resultCode == Activity.RESULT_OK) {
        activityViewModel.onCurrentNoteEdited()
    }
}
}

```

A similar change will occur for all the result launcher migrations, the main change would be to use `requireActivity()` since the fragment has to communicate with its host activity (`DeckPicker`).

(iii) In the `setupFlows()` function, there are 3 flows we have to migrate since they use the new M3 search view that has been migrated, the others would most likely crash since they use a text view. The 3 to be migrated are `cardSelectionEventFlow`, `flowOfSaveSearchNamePrompt`, `flowOfChangeNoteType`.

Current code in Card Browser:

```
viewModel.cardSelectionEventFlow.launchCollectionInLifecycleScope(::onSelectedCardUpdated)
```

Code after migration:

```
activityViewModel.cardSelectionEventFlow.launchCollectionInLifecycleScope(:onSelectedCardUpdated)
```

(iv) Shortcuts can be migrated to `DeckPicker` and they should be wrapped in an if block so that each destination's shortcuts are separate.

```
if (currentTab == BROWSER)
```

(v) Lifecycle overrides like `onResume` and `onPause` can be migrated to the fragment as follows:

Current code in Card Browser:

```
override fun onPause() {  
    super.onPause()  
    viewModel.removeUnsubmittedInput()  
}
```

Code after migration:

```
override fun onHiddenChanged(hidden: Boolean) {  
    super.onHiddenChanged(hidden)  
    if (hidden) {  
        activityViewModel.removeUnsubmittedInput()  
        view?.let { requireContext().hideKeyboard(it) }  
    }  
}
```

These are the main functions to migrate, the helpers can be moved to the fragment as needed.

6. Decoupling the Navigation Drawer

At the moment, AnkiDroid's main navigation tool is the navigation drawer which is outdated now and is to be replaced by the bottom navigation bar as seen in the above designs.

Currently, the navigation drawer code lies in `NavigationDrawerActivity.kt` which also acts as a base class for `DeckPicker.kt` and `CardBrowser.kt`, the issue is that since it is a parent for `DeckPicker` and `CardBrowser` it becomes harder to decouple it. I propose 2 phases to decouple the navigation drawer:

(i) For initial testing purposes, we can start by adding a flag in the drawer activity which wraps all the initialization and wiring. This can be set to false in the required child classes to skip the navigation drawer initialization entirely.

To be specific, we want to wrap `initNavigationDrawer()`, and the respective lifecycle methods to override the hamburger menu and remove it. Since, we are skipping the initialisation of the navigation drawer, it won't be instantiated in the `toolbar.xml` file and the linear layout will push the `FixedTextView` left accordingly:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.appcompat.widget.Toolbar
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/toolbar"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:background="?attr/appBarColor"
    android:minHeight="?attr/actionBarSize"
    android:theme="@style/ActionBarStyle"
<!-- we deleted the home as up indicator -->

    <com.ichi2.ui.FixedTextView
        android:id="@+id/toolbar_title"
        android:layout_width="wrap_content"
        android:layout_height="match_parent"
        android:textColor="@color/white"
        android:textSize="20sp"
        android:visibility="gone" />

    <Spinner
        android:id="@+id/toolbar_spinner"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```



```
        android:background="?attr/selectableItemBackground"
        android:visibility="gone"
        app:popupTheme="@style/ActionBar.Popup" />
    </androidx.appcompat.widget.Toolbar>
```

(ii) Once the bottom navigation bar is set, the drawer logic and its related xml and files can be removed. We can accomplish this by moving the non-drawer functionalities to the BottomNavActivity class, the functionalities are:

- (i) openCardBrowser()
- (ii) openStatistics()
- (iii) openSettings()

These are the routing functions that will handle swapping destination fragments. This is how they would look in the base class:

```
protected open fun onBrowserSelected() {
    deckPickerContentView.visibility = View.GONE
    showFragment(CardBrowserFragment(), "browser")
}
```

(iii) We can move the enablePostShortcut() method to DeckPicker.kt, there should be no issues with this since it is not related to [NavigationDrawerActivity.kt](#). One thing to be careful of here is regarding the line:

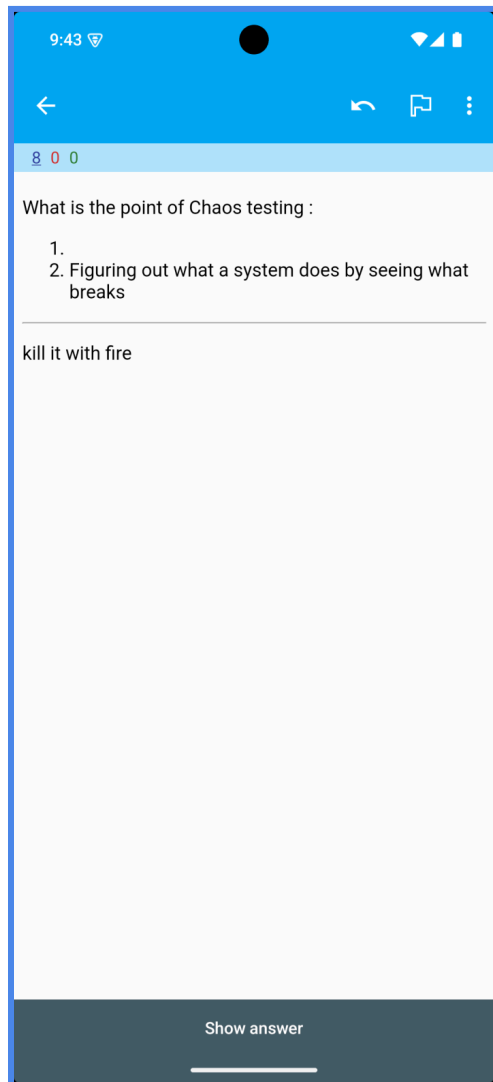
```
val intentCardBrowser = Intent(context, CardBrowser::class.java)
```

Since CardBrowser will become a fragment, we have to change the intent to DeckPicker and then pass the destination as onBrowserSelected() to show the browser fragment.

7. Reviewer Back Button

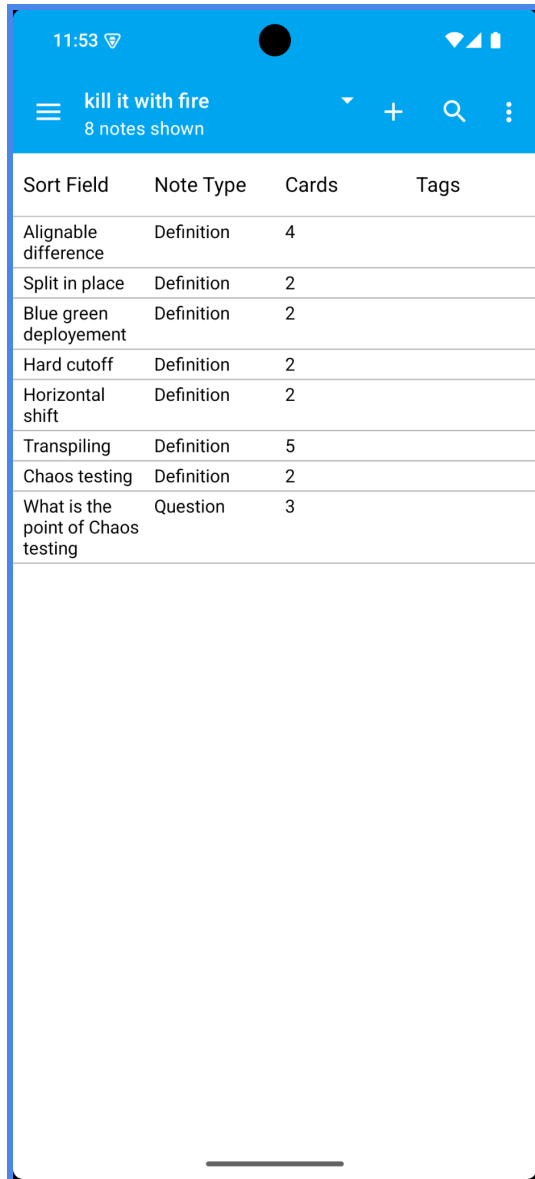
We can remove the hamburger button in the reviewer screen and replace it with a back button, the implementation is below:

```
drawerToggle.isDrawerIndicatorEnabled = false
```



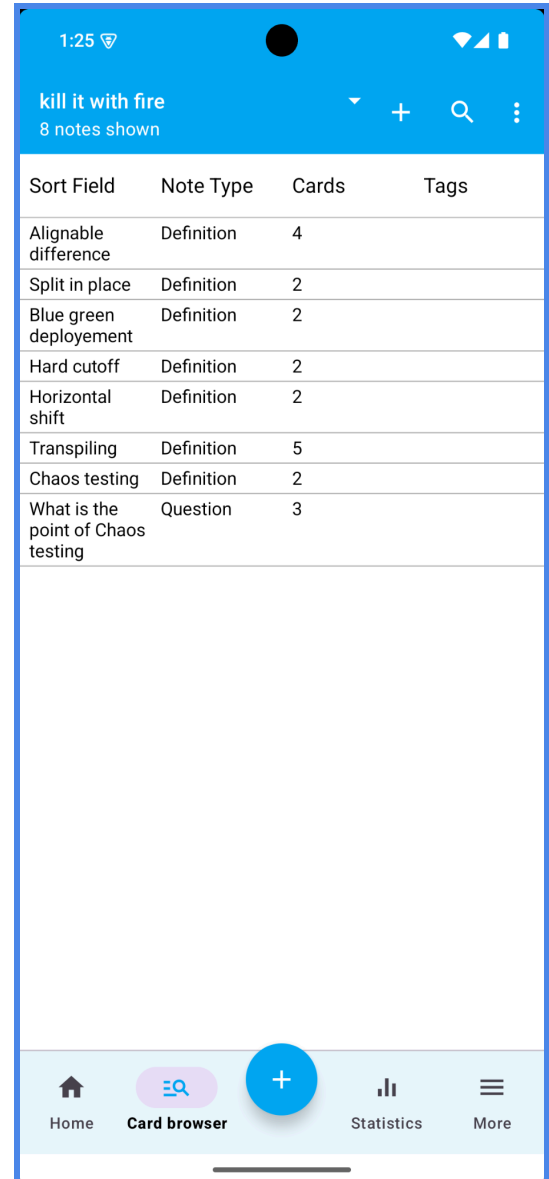
8. Card Browser & Statistics Design

This section of the proposal will detail how we can add the bottom nav bar to the card browser and statistics while keeping their functionality the same.



Sort Field	Note Type	Cards	Tags
Alignable difference	Definition	4	
Split in place	Definition	2	
Blue green deployment	Definition	2	
Hard cutoff	Definition	2	
Horizontal shift	Definition	2	
Transpiling	Definition	5	
Chaos testing	Definition	2	
What is the point of Chaos testing	Question	3	

Before

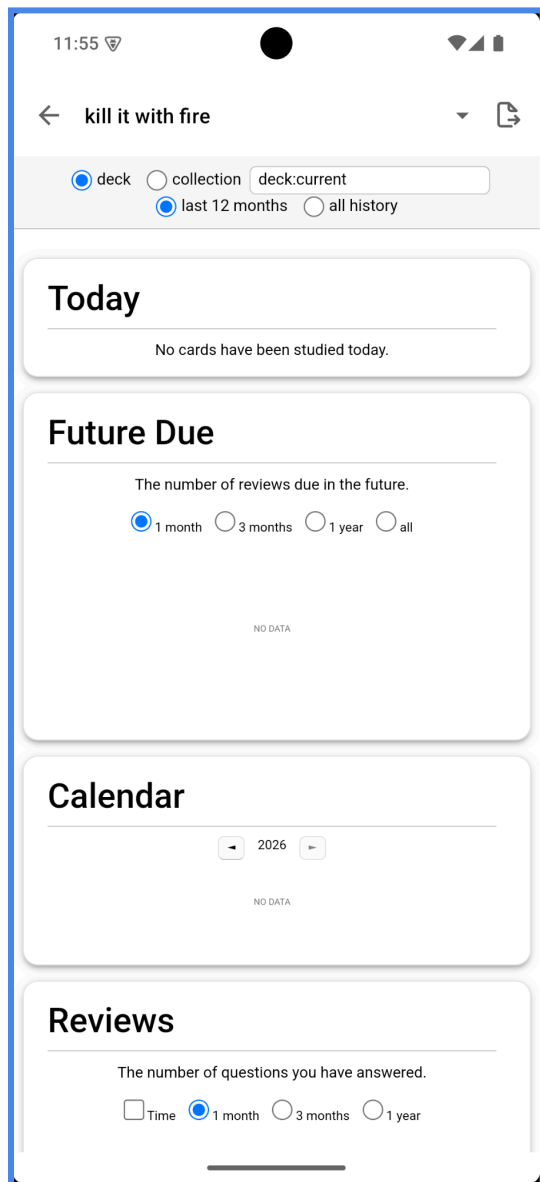


Sort Field	Note Type	Cards	Tags
Alignable difference	Definition	4	
Split in place	Definition	2	
Blue green deployment	Definition	2	
Hard cutoff	Definition	2	
Horizontal shift	Definition	2	
Transpiling	Definition	5	
Chaos testing	Definition	2	
What is the point of Chaos testing	Question	3	

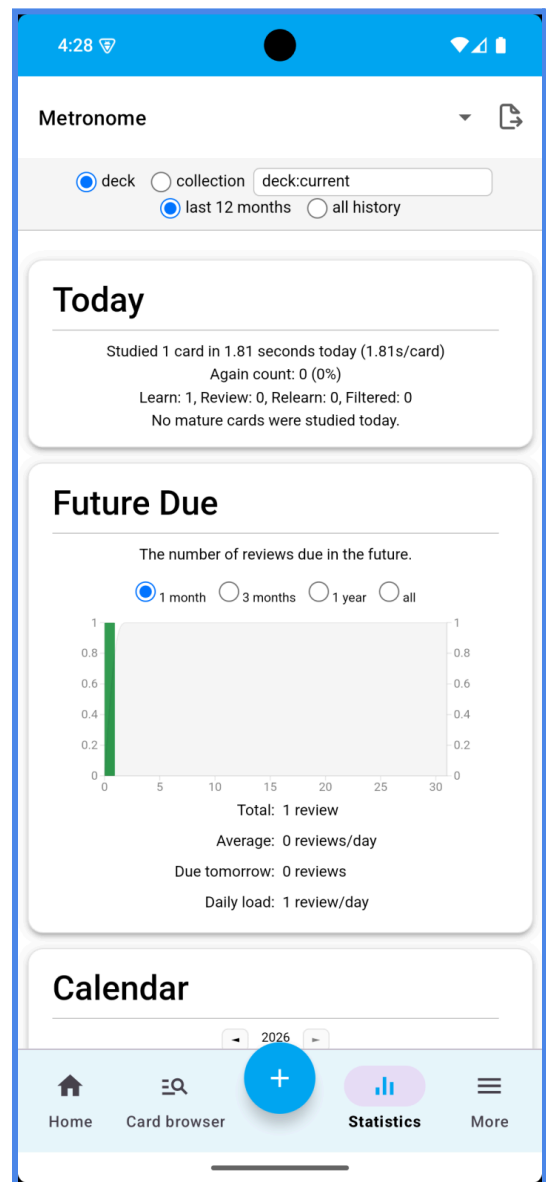
After

Persistence:

Since we will have the ability to move between destinations through the bottom nav bar, a good UX choice is to preserve the values of the user's current scroll position along with the card index so that when they move to another destination and return, the same scroll position can be restored. However, this shouldn't be a concern if we use fragments since state would be automatically preserved there since the fragment is never destroyed.



Before



After

The migration for this would be done by this point, the fragment would be shown along with the nav bar. The above is how it would look.

9. Keyboard Shortcuts

Earlier, we defined our base class (BottomNavActivity.kt), this makes it much easier to easily define keybinds for the bottom nav bar. The shortcuts⁴ we could use are:

Alt + 1 → Highlights Home Tab, triggers onHomeSelected()

Alt + 2 → Highlights Search Tab, triggers onBrowserSelected()

Alt + 3 → Highlights Stats Tab, triggers onStatsSelected()

Ctrl + N → Skips tab highlighting, triggers onAddSelected() (launches Speed Dial / Note Editor directly)

Implementation:

We can override dispatchKeyEvent() in our base class to implement the shortcuts, and then route the user to the respective destination.

Kotlin Snippet:

```
override fun dispatchKeyEvent(event: android.view.KeyEvent): Boolean {
    if (event.action == android.view.KeyEvent.ACTION_DOWN &&
        event.isAltPressed) {
        val handled = when (event.keyCode) {
            android.view.KeyEvent.KEYCODE_1 -> {
                navigationBar?.selectedItemId = R.id.action_home
                true
            }
            android.view.KeyEvent.KEYCODE_2 -> {
                navigationBar?.selectedItemId = R.id.action_card_browser
                true
            }
            android.view.KeyEvent.KEYCODE_3 -> {
                navigationBar?.selectedItemId = R.id.action_stats
                true
            }
            android.view.KeyEvent.KEYCODE_N -> {
                onAddSelected()
                true
            }
            else -> false
        }
        if (handled) return true
    }
    return super.dispatchKeyEvent(event)
}
```

⁴ Referred:

<https://discord.com/channels/368267295601983490/701922522836369498/1485306456902205470>

10. Tablet UI

For tablets, according to M3 guidelines⁵, we should use a navigation rail for tablets since a bottom bar would take up too much horizontal space.

This would require designing a separate rail and possibly linking it to the base class we develop for the bottom nav bar.

Due to the separate redesign needed, we can leave this as a stretch goal to implement if time permits.

An important thing to note is that during the fragment migration, we will leave all the classes that are needed for the tablet as they are i.e. navigation drawer and card browser will all be left alone and not deleted so that the tablet UI doesn't break since crucial elements like the resizable divider in the card browser are present in the activity. We can check if it's for the tablet or for the phone and launch accordingly (in deckpicker):

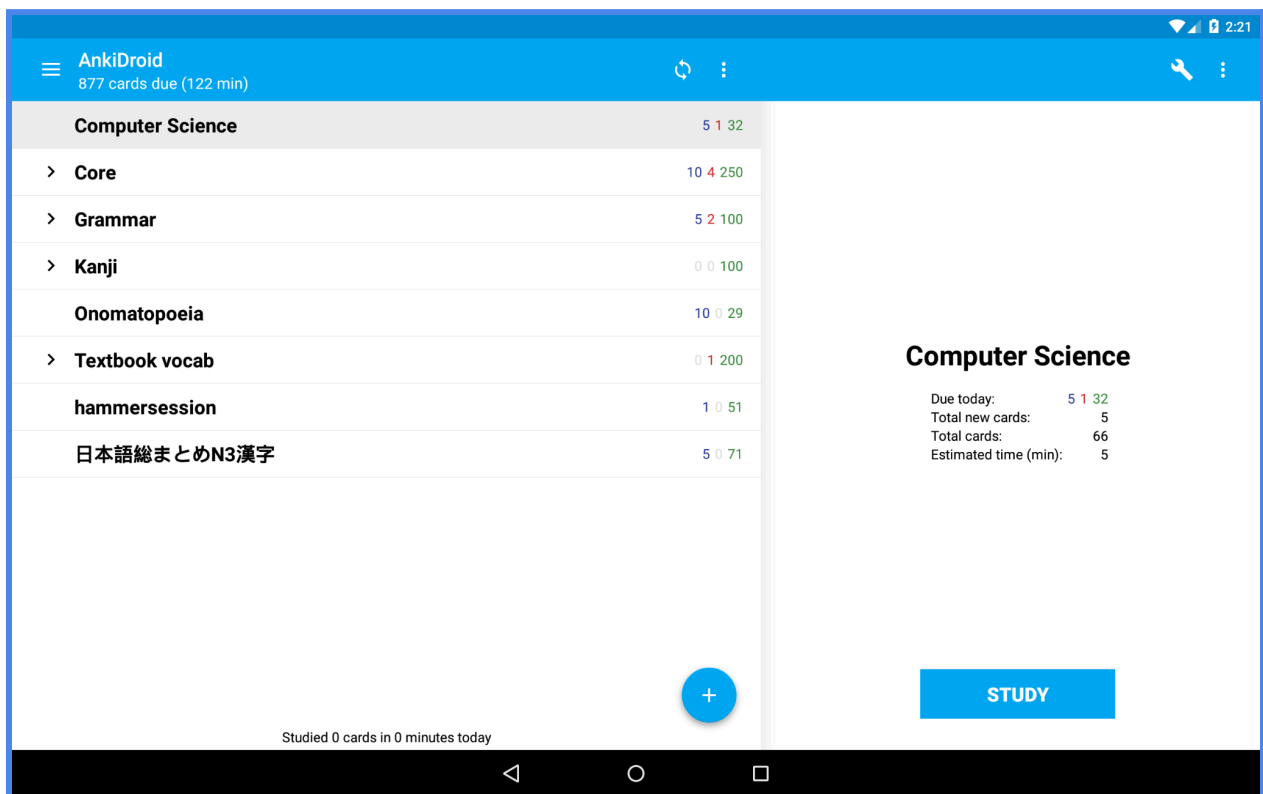
```
// Phone Route: The user clicks the bottom screen
private fun setupBottomNavigation() {
    val bottomNav =
    findViewById<BottomNavigationView>(R.id.bottom_navigation)
    bottomNav?.setOnItemSelectedListener { item ->
        when (item.itemId) {
            R.id.nav_browser -> {
                deckPickerContentView.visibility = View.GONE
                showFragment(CardBrowserFragment(), "browser")
                true
            }
            R.id.nav_stats -> {
                deckPickerContentView.visibility = View.GONE
                showFragment(Statistics(), "statistics")
                true
            }
            else -> false
        }
    }
}
```

⁵ <https://m3.material.io/components/navigation-rail/>

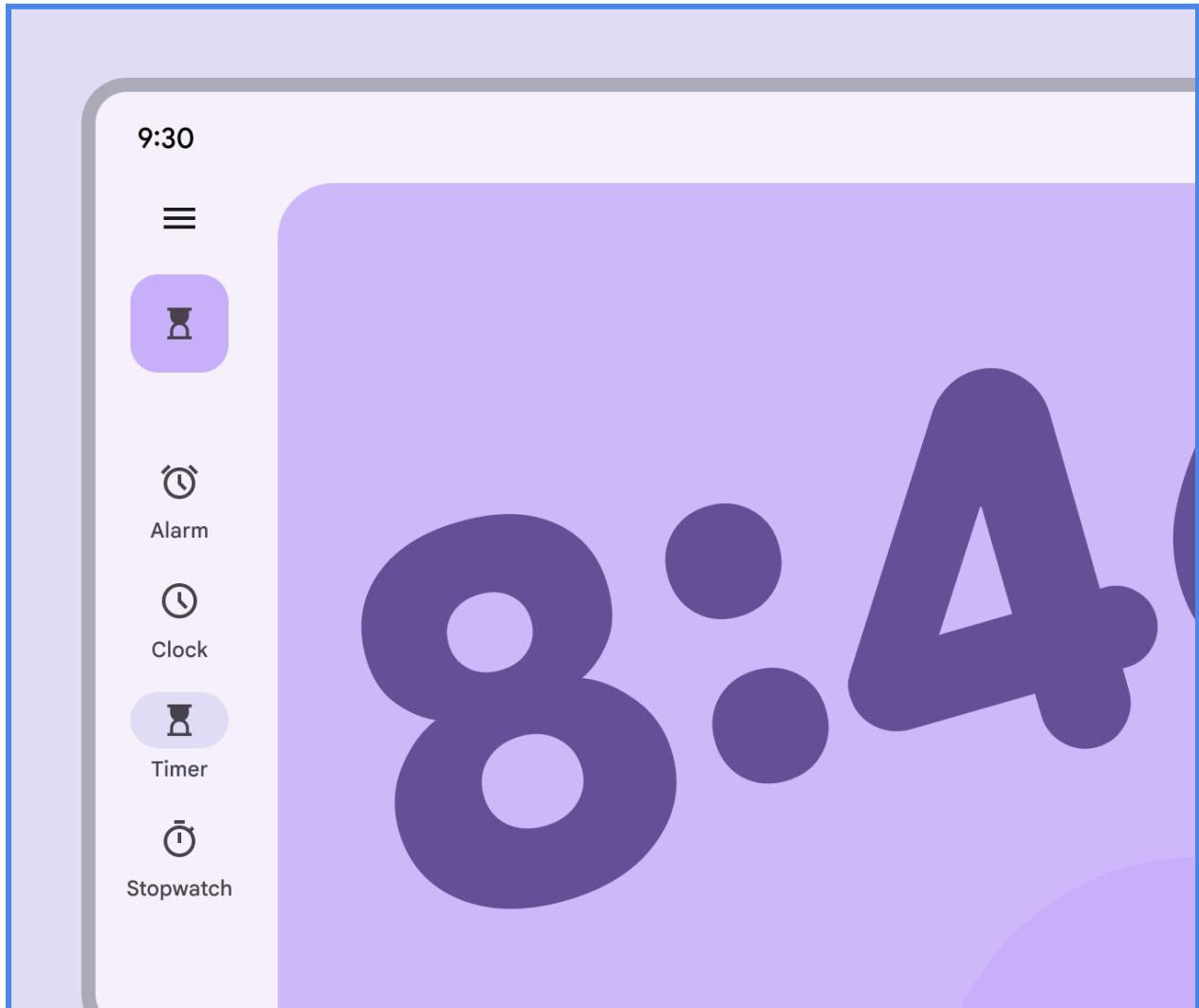
If the tablet is being used:

```
override fun onNavigationItemSelected(item: MenuItem): Boolean {
    when (item.itemId) {
        R.id.nav_browser -> startActivity(Intent(this,
CardBrowser::class.java))
        R.id.nav_statistics -> startActivity(Intent(this,
StatisticsActivity::class.java))
    }
    drawerLayout.closeDrawers()
    return true
}
```

Current Design:

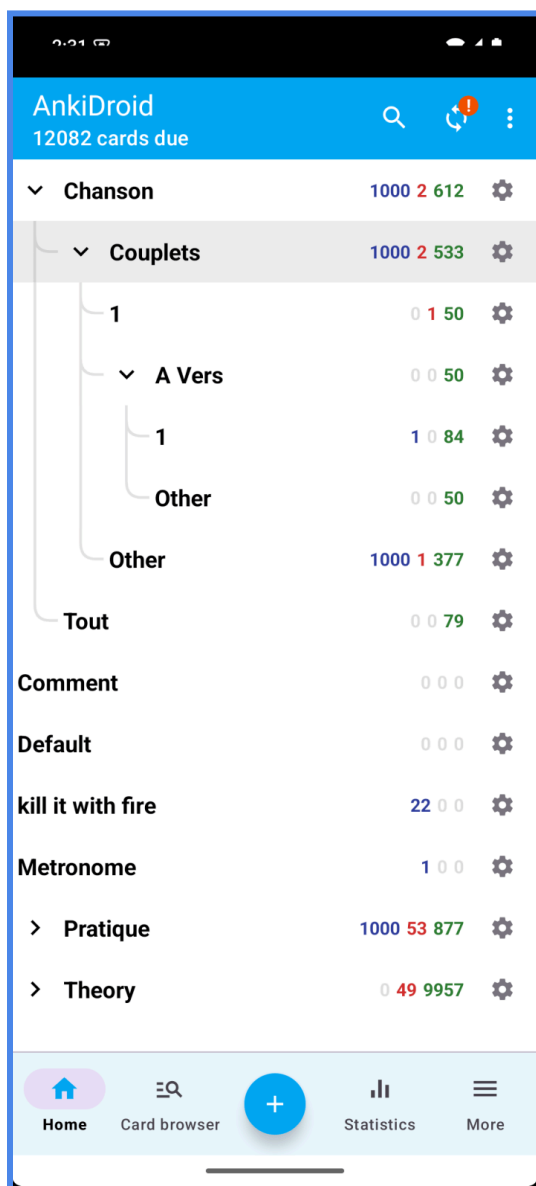


Example screenshot of Navigation Rail:



11. Deck Picker Redesign

The deck picker has long been a pain point for AnkiDroid users due to its cluttered look and issues with accessibility of options for power users. For this, I propose a change in its look and feel based on user sentiments⁶.



Changes to be made:

(i) Adding the 3 dots button to allow for easy access to deck options⁷

(ii) Adding the hierarchy lines to allow for a better subdeck look

(iii) Changing the deck menu from a floating one to a bottom sheet dialog when the user long presses

(iv) Improving the onboarding message when there are no decks

⁶ <https://www.reddit.com/r/Anki/comments/1s0zgx0/comment/obxcqe5/>

⁷ <https://www.reddit.com/r/Anki/comments/1s0zgx0/comment/oby3q9u/>

Implementation:

(i) Options Button:

The gear option menu as seen in the screenshot can be added to the decks via adding it to the deck_item.xml.

XML Snippet:

```
<ImageButton
    android:id="@+id/options_button"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_alignParentEnd="true"
    android:layout_centerVertical="true"
    android:background="?attr/selectableItemBackgroundBorderless"
    android:contentDescription="@string/study_options"
    app:srcCompat="@drawable/ic_settings_black"
    app:tint="?attr/colorOnSurfaceVariant" />
```

We can edit DeckAdapter.kt to route the deck options screen to the options button.

Kotlin snippet:

```
//listener
private val onDeckOptionsRequested: (DeckId) -> Unit,
.
.
.
//inside onBindViewHolder
val itemNode = getItem(position)
binding.optionsButton.setOnClickListener {
    onDeckOptionsRequested(itemNode.did)
}
```

We have to make the corresponding change in the deck adapter call in deckpicker.

Kotlin snippet:

```
deckListAdapter =
    DeckAdapter(
        this,
        onDeckSelected = { onDeckClick(it,
        DeckSelectionType.DEFAULT) },
        onDeckCountsSelected = { onDeckClick(it,
        DeckSelectionType.SHOW_STUDY_OPTIONS) },
```

```

        onDeckChildrenToggled = { deckId ->
            viewModel.toggleDeckExpand(deckId)
            dismissAllDialogFragments()
        },
        // the below line is changed
        onDeckOptionsRequested = ::showDeckPickerContextMenu,
        onDeckRightClick = { deckId, x, y ->
            showDeckPickerRightClickContextMenu(deckId, x, y)
            Timber.d("Right Click on deck recorded!! %d, %f %f",
deckId, x, y)
        },
    )

```

User Request:

3. The long press dialog!!! You know? the one with the million options where you can never find the "deck options".

(ii) Hierarchy lines:

We can add hierarchy lines to subdecks to improve visual clarity for users. This was a request made in <https://www.reddit.com/r/Anki/comments/1s0zgxo/comment/obxcqe5/>.

This is similar to what is done above, when a new subdeck is created we can draw a hierarchy line next to it that leads from the upper level.

Currently, in DeckAdapter.kt, the following line is responsible for indenting the subdecks:

```
indent.minimumWidth = nestedIndent * node.depth
```

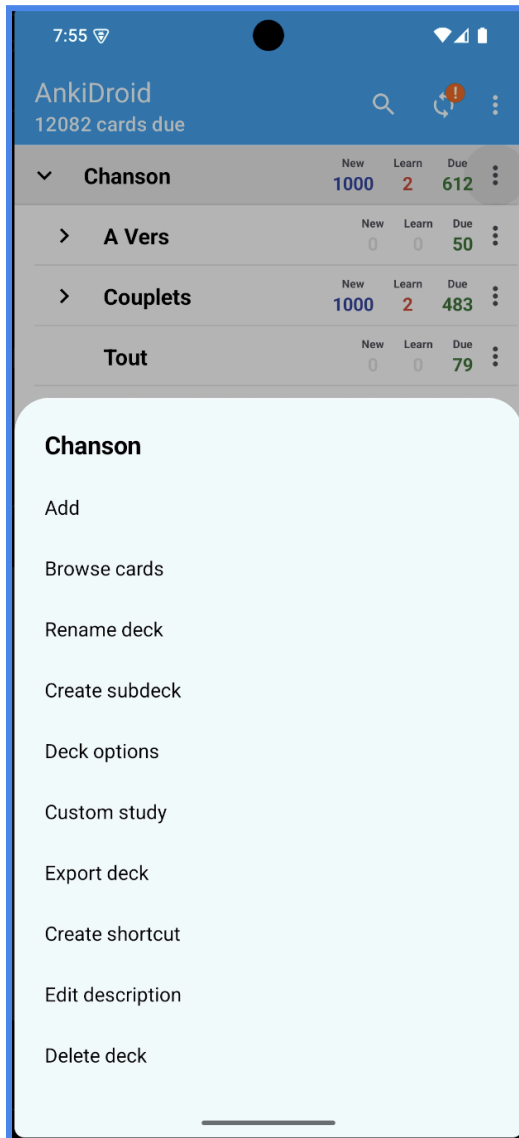
We can use the depth of a node to perform calculations that tell us how long the drawn line should be.

```
val targetWidth = (depth * nestedIndent).toInt()
```

(iii) Bottom Dialog Sheet:

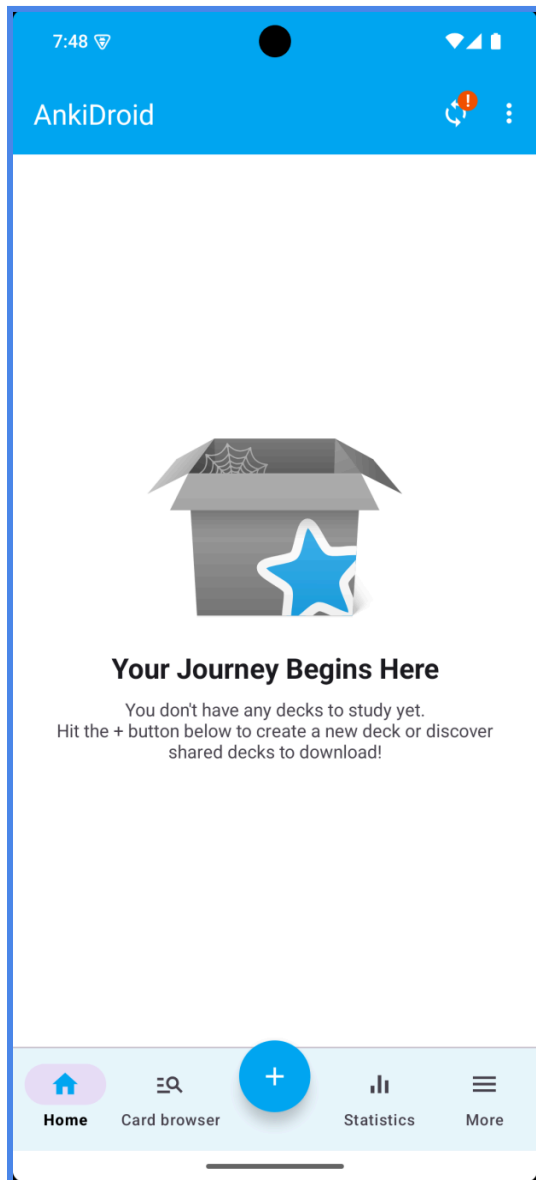
For the bottom sheet dialog conversion, we can refactor `DeckPickerContextMenu.kt` and make it extend `BottomDialogSheet` along with the line:

```
val dialog = BottomSheetDialog(requireContext(), theme)
```



(iv) Onboarding message:

When a user has no decks, a basic message appears that tells them the same. For new users, seeing AnkiDroid for the first time is overwhelming and since the empty deck picker screen is the first one they will see, we can add an onboarding message.



Currently, the message string is stored in the `res/layout/deck_picker.xml` file. We can change the string resource accordingly along with the typography in the XML.

XML Snippet:

```
<LinearLayout

    android:id="@+id/no_decks_placeholder"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center_vertical"
    android:orientation="vertical">

    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="24dp"
        android:layout_gravity="center_horizontal"
        app:srcCompat="@drawable/anki_box" />

    <com.ichi2.ui.FixedTextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="8dp"
        android:gravity="center"
        android:text="@string/no_cards_placeholder_title"
        style="@style/TextAppearance.AppCompat.Medium" />

    <com.ichi2.ui.FixedTextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textAlignment="center"

    android:text="@string/no_cards_placeholder_description"
        style="@style/TextAppearance.AppCompat.Body1" />
</LinearLayout>
```

Success Criteria:

- 1) Successfully instantiate all the required XML's
- 2) Migrate the required activities to fragments
- 3) Link all the fragments to the nav bar
- 4) Decouple the navigation drawer in all the destination screens and replace with the new base class
- 5) Implement keyboard shortcuts that link with the navigation bar
- 6) Redesign the main deck picker screen

Timeline:

Phase	Time Period	Details	Est. Hours
N/A	May 06 - May 15	University Exams	N/A
Community Bonding Period	May 15 - May 24	1) Discuss architectural approach and designs 2) Finalise Success criteria for the project 3) Familiarising myself with the relevant parts of the codebase	N/A
Phase 1	Week 1 May 25 - May 31	1) Setup dev environment 2) Scaffold the BottomNavActivity base class 3) Write the xml wrapper for the nav bar (activity_bottom_nav.xml)	12
	Week 2 June 01 - June 07	1) Finish the bottom_nav_menu.xml and related designs 2) Migrate Card browser	20
	Week 3 June 08 - June 14	1) Remove navigation drawer dependencies 2) Route fragments in base class	12

	Week 4 June 15 - June 21	1) Migrate Card Browser to using nav bar XML and base class 2) Migrate statistics 3) Integrate both to the nav bar	20
	Week 5 June 22 - June 28	Testing and review period	10
	Week 6 June 29 - July 06	Buffer period	11
July 06 - July 10		Submit Midterm Evaluation	85
Phase 2	Week 7 July 7 - July 13	1) Look to optimise the current add fab animation logic 2) Discuss potential design with mentors and implement the same	10
	Week 8 July 14 - July 20	1) Look into deck picker code and xml 2) Change the onboarding message	12
	Week 9 July 21 - July 27	1) Implement the options button for all decks 2) Finish keyboard shortcuts for Nav Bar	10
	Week 10 July 28 - August 03	1) Implement hierarchy lines in the deck picker 2) Implement bottom sheet dialog (if discussed)	15
	Week 11 August 04 - August 10	Testing and review period	10
	Week 12 August 11 - August 17	Finish pending work & work on stretch goal - Tablet UI, Drag & Drop deck	20
	Week 13 August 18 - August 24	Buffer period	8
August 25 - August 31		Submit Final Evaluation	175

Myself:

My personal experience with Anki/AnkiDroid:

It was around 2018 when I was first introduced to Anki and the concept of SRS. While my 8th grade self was looking for ways to study, I came across a video by Ali Abdaal which said that Spaced Repetition is one of the best ways to learn and AnkiDroid is one of the best apps to practice the method. I've been inconsistently using it since then, hoping it can solve my exam problems.

What I expect to get from participating in GSoC

I don't think anything can compare to the feeling of shipping a working feature - a feeling that I've been experiencing a lot since I started working on open source and AnkiDroid specifically. The changes I've made, however, feel smaller compared to what I feel I can really do and I believe GSoC is the perfect way to leave a lasting impact that I can be proud of. I'd also love to extract the knowledge from my mentors as much as possible.

I feel learning from your seniors is the best way to grow quickly, and I hope with the knowledge they impart, I can help out the community that's helped me out so much over the past few months.

Programming languages and tools I know

Kotlin, Java, C++, Python, SQL, JSON, XML

Example of skills you taught yourself

While working on AnkiDroid over the past few months, I've learnt Git to an extent where I can confidently use it to push, pull, rebase, bisect. I've taught myself Kotlin, and while I'm not at the level of building complete apps by myself, I can definitely do a lot using the Android developer documentation. Unit tests are another thing I learnt with some guidance, though there's a lot left to learn. Learning and using google docs has been a frustrating experience over the past month but documentation is always very important and it's something I'm quite good at now.

Anything else:

Over the past 3 months, I've truly enjoyed all the work I've done to help AnkiDroid and coincidentally I've worked on some heavy user-facing issues, completing them has been the most rewarding thing I've done since I decided to go down the CS route. Over the course of the project, I'll definitely give it my full focus and make sure it gets completed on time while communicating with mentors frequently. Finally, the learning I'd gain through GSoC would help me give back to the AnkiDroid community in a much quicker way and hopefully ease burdens of the current reviewers/maintainers. This is a post note after completing my proposal, I hope the proposal has reflected how much effort I've put into it as well as the mentors for giving me advice. I've been burnt out and close to giving up several times while writing this but having finished this now has been so rewarding so I'm thankful for this experience and hopefully the experience to implement this in AnkiDroid.